NTNU
Kunnskap for en bedre verden

Department of Computer Science

TDT4240 - Software Architecture

Requirements Document

Student ID - Author:

151315 - Yannic Neu

163719 - Niklas Wichter

163609 - Helena Freya Dingeldein

163739 - Abhisek Kumar

114440 - Supattra Wongsamal

564949 - Peter Olai Johnsen

Quality Attributes: Modifiability, Usability and Performance

16.02.2026

Table of Contents

List of Figures i

List of Tables i

1 Project Summary 1

2 Introduction 1

2.1 Project Description 1

2.2 The Game Concept 1

3 Functional Requirements 2

4 Quality Requirements 3

4.1 Modifiability 4

4.2 Usability 4

4.3 Performance 5

5 COTS Components and Technical Constraints 5

6 Issues 5

7 Changes 6

8 Individual Contributions 6

List of Figures

1 Battleship board 1

List of Tables

1 Functional Requirements for Battleships-EX 3

2 M1: Adding a New Action Card 4

3 M2: Changing Board Size 4

4 M3: Swapping Backend Provider 4

5 U1: Learning the Game on First Play 4

6 U2: UI Feedback Responsiveness 4

7 U3: Understanding Action Cards 5

8 P1: Online Turn Synchronization 5

9 P2: Loading an Ongoing Match 5
10 P3: Processing Game Logic 5
11 Section Responsibilities 6

1 Project Summary

Project Name: Battleships-EX

Group Number: 10

Chosen COTS:

Primary quality attribute chosen: Modifiability

Secondary quality attribute: Usability, Performance

2 Introduction

2.1 Project Description

The following document presents the requirements for the "Battleships-EX" game project. It includes a concise description of the game concept, a specification of the functional and developmental requirements, and the identification of architecturally significant quality attributes. Furthermore, the document addresses technical constraints and the selection of Commercial Off-The-Shelf (COTS) components, such as the libGDX framework and Firebase.

The requirements phase serves as the foundation of the project and forms the basis for the remainder of the development process. By establishing a clear set of functional requirements and quality attributes, we aim to identify potential challenges and constraints related to the software architecture early in the lifecycle. This document ensures a shared understanding among stakeholders, including end users, developers, and course staff, regarding the project's scope and foundational goals.

2.2 The Game Concept

Figure 1: Battleship board

Battleships-EX is a 2D, turn-based multiplayer game for Android that reimagines the classic naval combat tabletop game (see Figure 1) with a card-driven twist. While the core objective remains sinking the opponent's fleet by guessing their coordinates, our version introduces a resource management layer that enhances gameplay by adding strategic depth through Action Cards, as well as real-time interaction feedback through UI synchronization. This allows each player to see which cell the opponent is aiming at before the move is finalized.

The game session is organized into two distinct stages: initialization (lobby creation and in-game setup) and the repeating core gameplay loop, which continues until one player wins.

Stage 1: Initialization (Setup)

Lobby Creation: A player initializes a new session, selects an Action Deck consisting of Action Cards, and may configure specific match settings. Upon creation, the system generates a unique, random lobby code. The second player uses this code to join the session, establishing a synchronized connection via the Firebase Realtime Database backend.

Preparation Phase: Once the lobby is full, players enter the setup phase. Each player places a predefined assortment of differently sized ships onto their private 10×10 grid. The game transitions to the combat stage only after both players have confirmed their fleet positions.

Stage 2: Core Gameplay Loop

The core loop repeats until a "Last Man Standing" is determined.

The Turn (Combat Phase): At the beginning of each round, players receive a random Action Card drawn from the Action Deck. During their turn, players must decide whether to perform a standard attack or spend energy to play an Action Card that modifies the game rules. These cards enable strategic moves such as protecting a ship with a **Shield**, performing a **Double Shot**, or using **Scan** to reveal ships in a 3×3 area.

While the active player is selecting which cell to target, the opponent can see the intended cell in real time before the action is confirmed. This increases tension and introduces an additional strategic layer, particularly when combined with the Action Card system.

Resolution: To ensure consistency and prevent manipulation, all actions and card executions are validated on the server. The system calculates the outcome (Hit, Miss, or Card Effect), updates the global game state, and synchronizes the results across both clients for immediate visualization.

3 Functional Requirements

The following section presents the functional requirements for Battleships-EX. These requirements define the core functions of the system and aim to enhance the overall user experience. They help the group stay aligned throughout the development period and assist in determining which requirements have higher priority than others. The table does not represent the estimated development time.

Each functional requirement is stated with a corresponding ID and closely related requirements share the same main functionality category. Each requirement is given a priority level: high, medium, or low.

High: States that the functional requirement is absolutely necessary in order to make the game function, or is laid out as a requirement in the project specification.

Medium: The functional requirement is important, but is not essential for a functional application.

Low: The functional requirement is not required for game function, but is a requested feature to implement.

ID	Requirement(s)	Priority
FR1: Session Management		
FR1.1	The system shall allow users to create a session and generate a unique lobby code.	High
FR1.2	The system shall allow a second player to join the session using the valid lobby code.	High
FR1.3	The system shall allow a player to re-join a game session in progress.	High
FR1.4	The system shall automatically wipe a game session if there is no activity within a set time-frame to conserve server resources.	High
FR2: Core Gameplay & Input		
FR2.1	The game shall support touch controls for all player interactions	High
FR2.2	Players must be able to place their ships on a 10x10 grid before combat begins	High
FR2.3	The System shall support turn-based combat where players choose specific opponent tiles to attack	High
FR3: Action Card System		
FR3.1	The system shall deal one Action Card to a player at the beginning of their round	High
FR3.2	Players shall be able to play an Action Card during their turn by spending the required energy/resources	High
FR3.3	The server shall handle the validation and execution of all attacks and card effects to prevent manipulation	High

Table 1: Functional Requirements for Battleships-EX

4 Quality Requirements

We have chosen usability and performance as our secondary quality attributes, and modifiability as the primary quality attribute. Quality attributes are used to measure properties that indicate how well the system meets stakeholder needs. This section is intended to improve stakeholders' understanding of the system's quality characteristics.

Quality attributes should be considered continuously from a holistic perspective, including during the design, implementation, and deployment processes. A quality attribute scenario can be divided into six main parts:

- **Stimulus:** An event that arrives at the system or project.
- **Source of stimulus:** A human, computer system, or other actor that generates the stimulus.
- **Artifact:** The part of the system (or the entire system) that is affected by the stimulus.
- **Environment:** The conditions under which the stimulus occurs (e.g., normal operation, overload, etc.).
- **Response:** The activity that occurs as a result of the stimulus.
- **Response measure:** A measurable criterion used to evaluate whether the response satisfies the requirement.

4.1 Modifiability

Table 2: M1: Adding a New Action Card

Source	Developer
Stimulus	Add a new Action Card with a unique effect
Artifacts	Rules Engine, Card System
Environment	Design time
Response	Implement a new card class and register it in the Rules Engine
Response Measure	$\leq$ 2 modified classes, $\leq$ 3 new classes, completed in 1 person-day

Table 3: M2: Changing Board Size

Source	Developer
Stimulus	Modify the game board size (e.g., 10×10 to 12×12)
Artifacts	Rules Engine, UI Layout
Environment	Design time
Response	Update board configuration and UI scaling logic
Response Measure	$\leq$ 3 modules affected, completed in 1 person-day

Table 4: M3: Swapping Backend Provider

Source	Developer
Stimulus	Replace Firebase with a REST-based backend
Artifacts	Data Source Layer
Environment	Design time
Response	Implement new DataSource modules
Response Measure	No UI or domain changes; CI tests pass

4.2 Usability

Table 5: U1: Learning the Game on First Play

Source	End user
Stimulus	User plays the game for the first time
Artifacts	Touch UI, Tutorial
Environment	Runtime
Response	User follows the onboarding tutorial
Response Measure	User understands ship placement and turns within 3 minutes

Table 6: U2: UI Feedback Responsiveness

Source	End user
Stimulus	User interacts with the game grid (tap, drag, select)
Artifacts	UI, Game Grid
Environment	Runtime
Response	Display highlight or animation feedback
Response Measure	Visual response $\leq$ 150 ms

Table 7: U3: Understanding Action Cards

Source	End user
Stimulus	User selects an Action Card
Artifacts	Card UI Panel
Environment	Runtime
Response	Show card description and effect preview
Response Measure	$\geq$ 90% of test users use the card without assistance

4.3 Performance

Table 8: P1: Online Turn Synchronization

Source	Player
Stimulus	Player submits a turn (attack or card effect)
Artifacts	Firebase Realtime Database
Environment	Normal operation
Response	Synchronize updated state to opponent
Response Measure	End-to-end delay $\leq$ 400 ms (95th percentile)

Table 9: P2: Loading an Ongoing Match

Source	Player
Stimulus	User opens or resumes a match
Artifacts	Local Cache + Backend
Environment	Normal operation
Response	Load and display game state
Response Measure	Match interactive within $\leq$ 800 ms

Table 10: P3: Processing Game Logic

Source	Game Engine
Stimulus	Evaluate hit/miss or Action Card effect
Artifacts	Rules Engine
Environment	Runtime
Response	Compute result locally
Response Measure	Computation time $\leq$ 50 ms

5 COTS Components and Technical Constraints

The system uses Firebase Realtime Database as a COTS backend solution to handle real-time synchronization and persistence with minimal development effort. The application targets Android devices (API level 29 or higher). The design assumes stable internet connectivity and aims for low-latency updates during gameplay. Synchronization, storage, and concurrent sessions are limited by Firebase quota and platform restrictions.

6 Issues

We did not encounter severe issues yet, since we are still in the early stages of development.

7 Changes

This section is reserved for documenting changes that will occur in the future.

8 Individual Contributions

Section	**Responsible**
Introduction	Supattra
Introduction to each part	Supattra
Game Concept	Peter
Functional Requirements	Peter
Quality Requirements	Abhisek
COTS Components and Technical Constraints	Niklas

Table 11: Section Responsibilities